KALASINGAM ACADEMY OF RESEARCH AND EDUCATION

SQL Query Generator from Plain English

Project ID: PRJ008

Team ID: T008

Team GitHub Repository: github.com/hackwithprem/SQL-Query-Generator

Student Name	Registration Number	Individual GitHub Link
P.Prem Kiran	99230040709	github.com/hackwithprem
K.Kakumanu Bharagava Ram	99230040583	github.com/kakumanubhargavaram-rgb
K.Rakesh	99230040627	github.com/Rakeshkudirella
M.Eswar Ganesh	99230040378	github.com/muthukurie-ux

Project Trainer / Guide

Mr. SHIRWIN PRINCE I

Table of Contents

1. Introduction.....Section 1
2. Problem Statement & Objectives.....Section 2
3. System Architecture (RAG Pipeline).....Section 3
4. Implementation Details & Tech Stack.....Section 4
5. Results and Testing.....Section 5
6. Conclusion & Future Scope.....Section 6

1. Introduction

Structured Query Language (SQL) is widely used to retrieve and analyse information stored in relational databases. However, users need to understand SQL syntax, database tables, columns, joins, filtering, grouping, and aggregation to write queries correctly. SQL Query Generator from Plain English addresses this difficulty by allowing users to describe the information they need in natural language. The application combines a React interface, a FastAPI backend, a Groq-hosted language model, and BM25-based Retrieval-Augmented Generation (RAG) to prepare SQL queries that can be reviewed and executed against a compatible data source.

2. Problem Statement & Objectives

Problem: People who know what information they want may not know how to express it as SQL. Differences in database schemas and SQL dialects make query writing more difficult. Direct LLM output can also contain incorrect table or column names or unsafe statements, so generated SQL needs relevant context and backend validation.

Objectives:

- Convert plain-English questions into SQL using a configured language model.
- Use the selected database schema to guide table and column selection.
- Retrieve relevant SQL patterns from a curated knowledge base using BM25-based RAG.
- Validate generated SQL and reject prohibited operations and multiple statements.
- Support query execution, explanation, correction, file uploads, database connections, authentication, and query history.

3. System Architecture (RAG Pipeline)

The system combines two connected workflows: preparation of a reusable SQL-example knowledge base and processing of each user question. During query processing, the backend obtains the selected schema when available, retrieves relevant examples with BM25, sends the combined context to the Groq model, and validates the generated SQL before returning it to the user.

3.1 Knowledge Base Preparation & BM25 Indexing

The project maintains 45 curated natural-language questions paired with SQL examples in the backend knowledge_base.py module. When retrieval is first requested, the pure-Python BM25Index tokenizes each example question together with its category and builds an in-memory index. It stores term frequencies, document lengths, document frequencies, and inverse document frequency values. The index is cached by RAGService for reuse by later requests in the same backend process.

3.2 Query & Generation Phase

When a user submits a question, SQLGeneratorService asks RAGService to retrieve relevant examples if RAG is enabled. BM25 ranks the examples by lexical relevance and returns up to the configured top K results that meet the minimum score. The backend formats those examples and adds them to the model prompt alongside the user question and, when selected, the actual database schema. Groq generates a SQL candidate. A separate validator checks the candidate before it is returned; execution occurs only after a separate execution request.

4. Implementation Details & Tech Stack

Component	Technology Used	Purpose
Programming Language	Python	Core backend services and application logic.
Frontend	React 18, Vite 5, Tailwind CSS, Axios	User interface, forms, result display, and API requests.
Backend API	FastAPI, Uvicorn, Pydantic	REST endpoints, request validation, and service orchestration.
Large Language Model	Groq API; openai/gpt-oss-120b	Generates SQL and supports explanation and correction.
RAG Retrieval	Pure-Python BM25 / Okapi BM25	Ranks relevant examples from the 45-example SQL knowledge base.
Database & Data Tools	SQLAlchemy, sqlparse, SQLite, Pandas, openpyxl; optional Supabase	Schema access, SQL validation/execution, CSV/Excel handling, and configured storage/auth services.

4.1 Sample BM25 Configuration

The following settings enable lightweight retrieval and control how many reference examples are included in the prompt. The BM25 scoring implementation uses k1 = 1.5 and b = 0.75.

```
RAG_ENABLED = true
RAG_TOP_K = 3
RAG_MIN_SCORE = 0.1
BM25 k1 = 1.5; b = 0.75
```

5. Results and Testing

The project documentation defines functional verification scenarios for the main application workflow. Testing checks include Groq provider selection and offline fallback, BM25 retrieval when relevant examples are found, operation with RAG disabled, schema-aware and schema-free SQL generation, rejection of prohibited SQL and multiple statements, execution against a compatible data source, SQL explanation and correction, and authenticated CSV/Excel upload and preview. A successful RAG retrieval is indicated by a non-empty retrieved-example list and the rag_applied response field. The supplied project material does not report a measured accuracy percentage, hallucination-reduction figure, or benchmark response time; therefore, no numerical performance claim is made here.

6. Conclusion & Future Scope

Conclusion: SQL Query Generator from Plain English combines a React-based interface, a FastAPI backend, Groq language-model inference, and a lightweight BM25 retrieval component. Relevant SQL examples and selected database schema information are supplied as prompt context, while backend validation and a separate execution workflow provide control over how generated SQL is used. The application reduces the need for users to write SQL manually while keeping generated queries reviewable.

Future Scope: Future improvements include expanding and versioning the curated SQL knowledge base with dialect-specific examples, measuring retrieval quality with precision and recall, and evaluating hybrid BM25-plus-embedding retrieval where paraphrase handling is needed. Further work can add query-plan inspection, stricter execution controls, automated tests for limits and selected columns, structured monitoring, persistent production storage, and a schema-aware query review panel.